

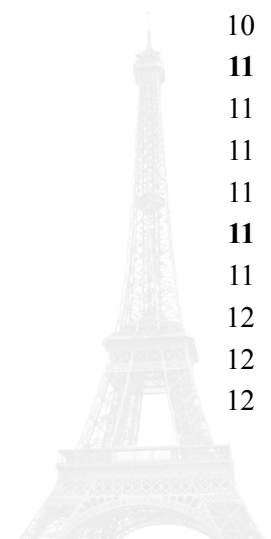
Knowledge Transfer: EdTech Multi-Agent LMS System



Version	Modified	By	Action
1.0	11/02/2026	Ashok Kumar M	Creation

Table of Contents

1. Backend Development – System Architecture	4
1.1 Overview	4
1.2 Core Framework	4
1.3 Orchestration Pattern	4
1.4 State Management	5
1.5 Observability & Debugging	5
2. API Integrations	5
2.1 Large Language Models (LLMs)	5
2.2 External Search Integration	6
2.3 Database Architecture	6
3. Data Models & Schema	6
3.1 Users	6
3.2 Courses	7
3.3 Enrollments	7
3.4 Grades	7
3.5 Engagement Metrics	7
4. Authentication & Security	8
4.1 Authentication Mechanism	8
4.2 Environment Security	8
5. Error Handling & Monitoring	8
5.1 Global Exception Handling	8
5.2 Failure Tracing	8
6. General Project Knowledge	9
6.1 Development Environment Setup	9
Python Requirements	9
Dependencies	9
Environment Configuration	9
Database Initialization	10
7. Version Control & Development Practices	10
7.1 Repository Structure	10
7.2 Branching Strategy	10
8. Deployment Process	11
8.1 Containerization	11
8.2 Orchestration	11
8.3 CI/CD Support	11
9. Roles and Responsibilities – Agent System	11
9.1 Manager Agent	11
9.2 User Profile Agent	12
9.3 Course Management Agent	12
9.4 Learning Path Agent	12



9.5 Assessment Agent	12
9.6 AI Coach Agent	13
9.7 Sandbox Agent	13
9.8 Lesson Planning Agent	13
9.9 Report Agent	13
9.10 Admin Task Agent	14
9.11 Calendar Agent	14
9.12 Engagement Agent	14
10. Project Roadmap	14
Phase 1 – Current Status	14
Phase 2 – Future Enhancements	15
11. Known Limitations & Testing	15
11.1 ML Dependencies	15
11.2 Testing Strategy	15
Conclusion	16



1. Backend Development – System Architecture

1.1 Overview

The EdTech Agents Backend is designed as a **Multi-Agent Learning Management System (LMS)** built on a modern **Agentic Architecture**. The system enables intelligent orchestration of domain-specific AI agents to handle educational workflows efficiently.

1.2 Core Framework

- **API Framework:** FastAPI (high-performance asynchronous API handling)
- **Workflow Orchestration:** LangGraph (for multi-step agent workflows)
- **Agent Framework:** LangChain (tool usage, memory handling, and reasoning)

1.3 Orchestration Pattern

The system follows a **Hub-and-Spoke Architecture**:

- A central **Manager Agent** acts as the orchestrator.
- It analyzes user intent.
- It routes requests to specialized domain agents such as:
 - Assessment Agent
 - Course Management Agent
 - Learning Path Agent
 - Report Agent
 - AI Coach Agent

This ensures modularity, scalability, and separation of concerns.



1.4 State Management

- A **shared state object** maintains context across multi-agent interactions.
- Conversation memory is preserved.
- Ensures continuity throughout a learning session.

1.5 Observability & Debugging

- Integrated with **LangSmith** for:
 - Agent reasoning traceability
 - Debugging complex workflows
 - Performance monitoring
 - Error diagnostics

2. API Integrations

2.1 Large Language Models (LLMs)

The system is provider-agnostic and supports:

- OpenAI
- Google Gemini
- Groq (Llama 3)
- Local models via Ollama

This allows:

- Cost optimization
- Performance tuning



- Vendor flexibility

2.2 External Search Integration

- Tavily API is integrated for:
 - Real-time web search
 - Curriculum research
 - Fact-checking educational content

2.3 Database Architecture

- **ORM Layer:** SQLAlchemy
- **Development Database:** SQLite
- **Production Database:** PostgreSQL
- **Agent-to-Database Interaction:** SQLDatabaseToolkit enables natural language database queries.

3. Data Models & Schema

The database schema supports a comprehensive LMS structure:

3.1 Users

- Student, Teacher, Admin roles
- Role-based access control
- Profile and identification details

3.2 Courses

- Course titles
- Descriptions
- Categories
- Difficulty levels

3.3 Enrollments

- Student–Course linking table
- Progress tracking
- Completion status

3.4 Grades

- Assessment results
- Student-course mapping
- Score storage



3.5 Engagement Metrics

- Login activity
- Forum participation
- Time-series engagement tracking
- At-risk learner identification





4. Authentication & Security

4.1 Authentication Mechanism

- JSON Web Token (JWT) based authentication framework implemented.
- Token generation and validation infrastructure available.
- Final API endpoint integration pending.

4.2 Environment Security

- API keys and database credentials stored in environment variables.
- No hardcoded secrets.
- Configuration handled via `.env` file.

5. Error Handling & Monitoring

5.1 Global Exception Handling

- Middleware implemented at API layer.
- Captures agent execution failures.
- Prevents application crashes.
- Returns structured error responses.

5.2 Failure Tracing

- All agent failures logged in LangSmith.
- Includes:
 - Input data



- Internal reasoning steps
- Error state details

6. General Project Knowledge

6.1 Development Environment Setup

Python Requirements

- Python 3.11 or higher
- Virtual environment recommended

Dependencies

Major libraries:

- fastapi
- uvicorn
- langchain
- langgraph
- pydantic



All dependencies are listed in the [requirements.txt](#) file.

Environment Configuration

A [.env](#) file is required for:

- LLM provider API keys (OpenAI / Groq / Gemini)
- Tavily API key
- LangSmith API key



- Database configuration

Database Initialization

- Initialization script available.
- Creates schema.
- Seeds sample users and courses.
- Supports testing and demo environment.

7. Version Control & Development Practices

7.1 Repository Structure

- edtech_agents/
 - |— agents/ → Specialized agent implementations
 - |— main.py → Application entry point
 - |— requirements.txt → Dependency list
 - |— Dockerfile → Container configuration

Each agent is implemented in its own subdirectory under `agents/`.

7.2 Branching Strategy

- Use feature branches for development.
- Submit Pull Requests for merging into main branch.
- Perform code review before merge.



8. Deployment Process

8.1 Containerization

- Fully Dockerized application.
- Dockerfile provided.
- Ensures environment consistency.

8.2 Orchestration

- `docker-compose.yml` available.
- Spins up backend service.
- Supports persistent data volumes.

8.3 CI/CD Support

- Compatible with standard CI/CD pipelines (e.g., GitHub Actions).
- Supports:
 - Automated testing
 - Image build
 - Deployment workflows

9. Roles and Responsibilities – Agent System

Each agent handles a specialized LMS domain.

9.1 Manager Agent

- Orchestrates system workflows.

- Classifies user queries.
- Maintains conversation context.
- Routes tasks to specialized agents.

9.2 User Profile Agent

- Manages learner data.
- Tracks skills.
- Updates learning preferences.

9.3 Course Management Agent

- Creates and modifies course content.
- Manages modules and curriculum structure.

9.4 Learning Path Agent

- Generates personalized learning sequences.
- Adapts based on progress and goals.

9.5 Assessment Agent

- Creates quizzes.
- Grades submissions.
- Provides performance feedback.

9.6 AI Coach Agent

- Personalized tutoring.
- Explains complex concepts.
- Answers academic queries.

9.7 Sandbox Agent

- Manages code execution environment.
- Provides feedback on code experiments.

9.8 Lesson Planning Agent

- Assists teachers.
- Generates lesson plans with:
 - Objectives
 - Activities
 - Resources



9.9 Report Agent

- Aggregates performance data.
- Generates dashboards and analytics reports.



9.10 Admin Task Agent

- Records attendance.
- Manages gradebooks.
- Handles operational queries.

9.11 Calendar Agent

- Schedules classes.
- Manages events.
- Handles time-based queries.

9.12 Engagement Agent

- Monitors student activity logs.
- Identifies low engagement.
- Flags at-risk students.

10. Project Roadmap

Phase 1 – Current Status

- Core multi-agent infrastructure implemented.
- Functional agents:
 - Profile Agent
 - Assessment Agent

- Report Agent
- AI Coach Agent
- Learning Path Agent
- Integrated observability.

Phase 2 – Future Enhancements

- Autonomous feedback loops (proactive agent intervention).
- LTI (Learning Tools Interoperability) integration.
- Advanced real-time analytics dashboards.
- Improved agent collaboration workflows.

11. Known Limitations & Testing

11.1 ML Dependencies

- Advanced features (e.g., vector stores) require:
 - PyTorch or
 - TensorFlow

Ensure proper installation when enabling vector search.

11.2 Testing Strategy

- Comprehensive test suite available.
- Covers:
 - Agent tools



- Memory handling
- Routing logic
- Recommended to execute tests before deployment.

Conclusion

The EdTech Agents Backend System is a scalable, modular, multi-agent LMS architecture designed for extensibility and intelligent automation. The hub-and-spoke agent model, combined with strong observability and containerized deployment, provides a robust foundation for future AI-driven educational innovations.

